

Principles of Bayesian statistics

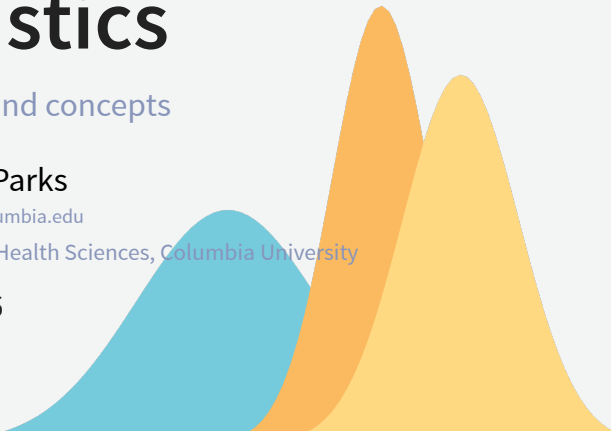
Key ideas and concepts

Robbie M. Parks

robbie.parks@columbia.edu

Environmental Health Sciences, Columbia University

Aug 5, 2026



Overview

Outline

- Overview
- Introduction to Bayesian methods and concepts
- Using **NIMBLE** for Bayesian inference
- A first look ahead to **R-INLA**

This course is for those who

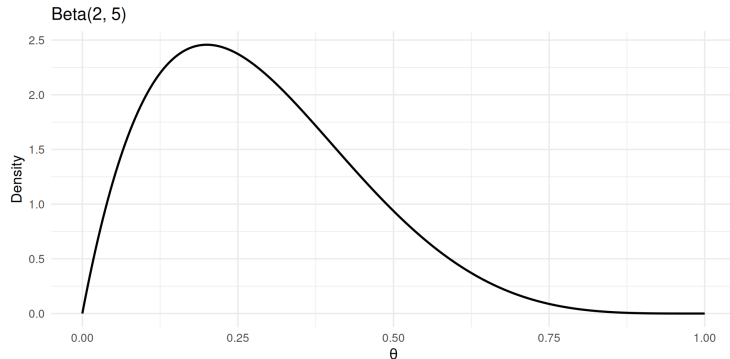
- Are interested in or who have heard about Bayesian modeling.
- Work in Environmental Health (or adjacent fields).
- Have little theoretical or practical experience of Bayesian ideas.
- Would like some tools and know-how to get started in an approachable and friendly setting.
- People who would like to have some fun while learning!
- There will be lots of code to get through over the next three days...

R

- R is an interactive environment developed by statisticians for data analysis.
- A more detailed Introduction to R can be found at <https://www.r-project.org>.
- R is the environment we will use throughout the workshop.
- But this isn't a course to learn R...
- There will be lots of code, though we will limit the equations to essentials.

R code output

- Output of code:



R code

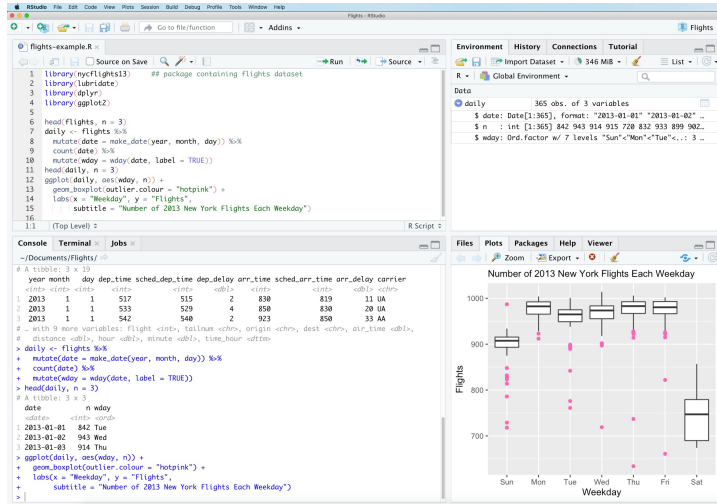
- Sample R code:

```
1 # Load packages
2 library(ggplot2)
3 # Beta(2, 5) density
4 theta <- seq(0, 1, length.out = 400)
5 dens <- data.frame(theta = theta, density = dbeta(theta, 2, 5))
6 # Plot
7 p <- ggplot(dens, aes(theta, density)) +
8   geom_line(linewidth = 1) +
9   labs(x = expression(theta), y = "Density",
10        title = "Beta(2, 5)") +
11   theme_minimal(base_size = 14)
12 plot(p)
```

RStudio by Posit

- RStudio is an integrated development environment (IDE) for R.
- RStudio provides a convenient graphical interface to R, making it more user-friendly, and providing many useful features.
- Such features include direct code execution, tools for plotting, history, debugging, and workspace management.
- A more detailed background on RStudio can be found at <https://posit.co/about/>.

RStudio by Posit



GitHub

- Version control to interface with **Posit (RStudio) Cloud**.
- This is just for information, as you will not be expected to be an expert on **GitHub** to participate in this **SHARP** course.
- However, you should learn it in general as it's really useful in today's modern research environment.



Posit (RStudio) Cloud

- **Posit Cloud** is a cloud-based **RStudio** which runs projects and code in the cloud.
- **Posit Cloud** allows convenient scaling and sharing of code, including for training programs such as SHARP.
- Registration for free is available via <https://posit.cloud/>.
- We will go through how to navigate **RStudio Cloud** together.
- More details are available via <https://docs.posit.co/cloud/>.

NIMBLE

- **NIMBLE** is one of several Bayesian inference packages.
- **NIMBLE** code is written in a slightly unusual format if you're used to just using base **R**.
- Written in the style of a program called **BUGS**, developed at Imperial College London.
- But you don't really need to know about **BUGS** to be able to use **NIMBLE**.

NIMBLE

- In the lab after this, we will start with some straightforward examples for situations you're likely familiar with to introduce the style of writing models.
- These examples will feature basic regression models using linear predictors.

R with NIMBLE (from lab immediately after this)

- As written in frequentist form:

```
1 model_freq <- lm(
2   y ~ x1 + x2 + x3,
3   data = df
4 )
```

R with NIMBLE (from lab immediately after this)

- Example basic linear multi-predictor regression:

$$y_i \sim \text{Normal}(\mu_i, \sigma) \quad i = 1, \dots, N$$

$$\mu_i = \alpha + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$$

R with NIMBLE (from lab immediately after this)

- Sample NIMBLE code:

```
1 code <- nimbleCode({
2   # priors for parameters
3   alpha ~ dnorm(0, sd = 100) # prior for alpha
4   beta1 ~ dnorm(0, sd = 100) # prior for beta1
5   beta2 ~ dnorm(0, sd = 100) # prior for beta2
6   beta3 ~ dnorm(0, sd = 100) # prior for beta3
7   sigma ~ dunif(0, 100) # prior for variance components
8
9   # regression formula
10  for (i in 1:n) { # n is the number of observations we have in the data
11    mu[i] <- alpha + beta1 * x1[i] + beta2 * x2[i] + beta3 * x3[i] # manual
12    y[i] ~ dnorm(mu[i], sd = sigma)
13  }
14 })
```

R with NIMBLE (from lab immediately after this)

- Comparing frequentist code....:

```
1 model_freq <- lm(
2   y ~ x1 + x2 + x3,
3   data = df
4 )
```

- ...with Bayesian (NIMBLE) code:

```
1 code <- nimbleCode({
2   # priors for parameters
3   alpha ~ dnorm(0, sd = 100) # prior for alpha
4   beta1 ~ dnorm(0, sd = 100) # prior for beta1
5   beta2 ~ dnorm(0, sd = 100) # prior for beta2
6   beta3 ~ dnorm(0, sd = 100) # prior for beta3
7   sigma ~ dunif(0, 100) # prior for variance components
8
9   # regression formula
10  for (i in 1:n) { # n is the number of observations we have in the data
11    mu[i] <- alpha + beta1 * x1[i] + beta2 * x2[i] + beta3 * x3[i] # manual entry of linear predictors
12    y[i] ~ dnorm(mu[i], sd = sigma)
13  }
14 })
```

INLA and R-INLA

- A preview of where we are heading later in the workshop:
- Integrated Nested Laplace Approximation (Rue, Martino & Chopin, 2009).
- We usually only want **posterior marginals**, not the joint posterior.
- A huge share of the models we actually fit belong to one restricted class — **latent Gaussian models**.
- Within that class, marginals can be *approximated deterministically*, accurately, and very fast.
- Don't worry about the details yet: we will encounter this properly from the end of tomorrow.

R-INLA

- Sample **R-INLA** code (~equivalent of earlier **NIMBLE** code).

```

1 # Fixed effect prior
2 prior_fixed <- list(
3   mean.intercept = 0,
4   prec.intercept = 1e-4,
5   mean           = 0,
6   prec           = 1e-4
7 )
8
9 # PC prior for sigma
10 prior_sigma <- list(
11   prec = list(prior = "pc.prec", param = c(100, 0.01))
12 )
13
14 formula <- y ~ x1 + x2 + x3
15
16 fit <- inla(
17   formula,
18   family   = "gaussian",

```

Introduction to Bayesian methods and concepts

MCMC and INLA

	MCMC	INLA
Approach	Stochastic sampling	Deterministic approximation
Model class	Almost anything	Latent Gaussian models only
Error	Monte Carlo error, $\rightarrow 0$ with time	Approximation error, fixed time
Runtime	Good! But can be a challenge to scale	Often much faster for large models
Reproducible	Up to the seed	Exactly
Diagnostics needed	Many	Few

Classical probability

- “Frequentist”.
- Limit of long-run relative frequency.
- Ratio of the number of times event occurred to the number of trials.

Classical probability

- Consider rolling a fair six-sided die many times.
- We are seeing how often we roll a 2.
- We roll die a large number of times (N).
- Probability that we roll a 2:

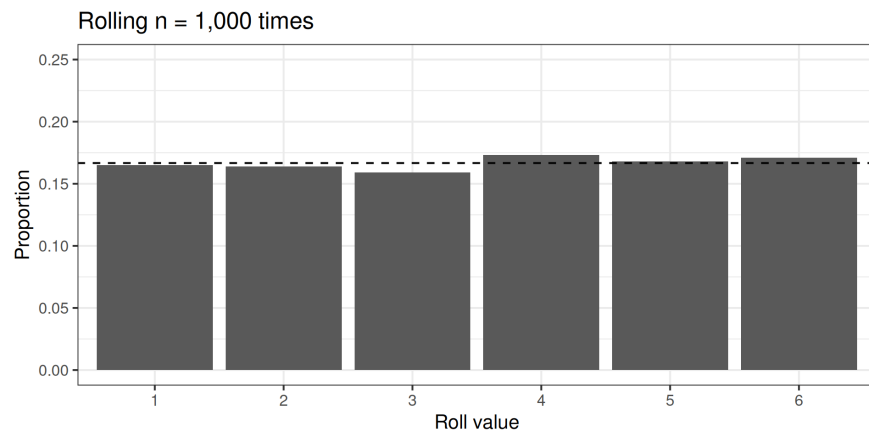


- $$Pr(X = 2) = N(X = 2)/N$$
- This will tend to $1/6 = 0.16666666...$ as $N \rightarrow \infty$

Classical probability

```
1 library(ggplot2)
2
3 set.seed(100)
4 rolls <- sample(1:6, 1e6, replace = TRUE)
5
6 for (n in c(1e2, 1e3, 1e4, 1e5, 1e6)) {
7   d <- data.frame(x = factor(1:6), p = tabulate(rolls[1:n], nbins = 6) / n)
8   print(
9     ggplot(d, aes(x, p)) +
10     geom_col() +
11     geom_hline(yintercept = 1 / 6, linetype = 2) +
12     coord_cartesian(ylim = c(0, 0.25)) +
13     xlab("Roll value") + ylab("Proportion") +
14     ggtitle(paste0("Rolling n = ", format(n, big.mark = ",", scientific =
15       theme_bw(base_size = 16)
16   )
17 }
```

Classical probability



Subjective probability

- What is it?
 - Probability as degree of belief, not long-run frequency.
 - Quantifies uncertainty based on prior knowledge, expert opinion, or experience.
- Key Features
 - Personal: Two people can assign different probabilities to the same event — both valid.
 - Coherent: Must follow the rules of probability (e.g., total = 1).
 - Updatable: Modified via Bayes' Theorem when new data arrives.
- Example
 - A researcher believes there's a 70% chance a new intervention reduces mortality, based on past studies and domain knowledge — even before seeing the data.

Thomas Bayes and Simon Pierre Laplace

- Started with these two.
- Bayes (1701-1761) is why it's called **Bayesian**.
- Sorry Laplace (1749-1825)!



Bayesian inference

- Leaving behind **frequentist** inference here.
- **Bayesian** when the prior is unknown and a distribution is specified for it.
- Bayes theorem exists in frequentist world, but here prior is a distribution.
- Prior + Data -> Posterior.
- Posterior can be thought of as **compromise** between prior evidence and current data.

Bayes theorem, prior, likelihood, and posterior

- $P(\theta|y) \propto P(y|\theta)P(\theta)$
- $P(\theta)$ is the **prior** (known or estimated a priori).
- $P(y|\theta)$ is the **likelihood** (probability of data given parameters of interest).
- $P(\theta|y)$ is the **posterior** (parameters of interest given data).

Bayesian inference



Priors (very briefly)

- Priors represent subjective influence.
- Priors also represent previous evidence.
- Prior choice can be vital.
- We will explore choosing priors in more detail at the end of today (Day 1).

Conjugacy (very briefly)

- When the posterior is an analytical solution of the prior and likelihood.
- Examples include:
 - Normal data and Normal prior.
 - Binomial data and Beta prior.
 - Poisson data and Gamma prior.

Normal-normal conjugacy example

Let:

- Observations: $y_1, \dots, y_n \sim N(\theta, \sigma^2)$, with known σ^2
- Prior: $\theta \sim N(\mu_0, \sigma_0^2)$
- Then the **posterior** is:

$$\theta \mid y \sim N(\mu_n, \sigma_n^2)$$

- where:

$$\sigma_n^2 = \left(\frac{n}{\sigma^2} + \frac{1}{\sigma_0^2} \right)^{-1}$$

$$\mu_n = \sigma_n^2 \left(\frac{n\bar{y}}{\sigma^2} + \frac{\mu_0}{\sigma_0^2} \right)$$

Non-conjugacy (most of the time in real world)

- Because most prior-likelihood-posterior relationships are non-conjugate.
- No closed-form posterior (unlike normal-normal example).
- Need to approximate posterior from sampling methods.
- Need non-analytical solution to infer distributions.
- This is where ‘brute force’ sampling of distributions takes place.

Obtaining brute force posterior distribution

- Many different types of Markov Chain Monte Carlo (MCMC):
 - **Metropolis-Hastings:**
 - Proposes moves; accepts with probability based on posterior ratio.
 - General-purpose, but can mix slowly in high dimensions.
 - **Gibbs Sampling:**
 - Samples each parameter conditionally.
 - Requires full conditional distributions.
 - Efficient when conditionals are tractable.
 - **Hamiltonian Monte Carlo (HMC):**
 - Uses gradient information to propose informed moves.
 - Efficient in high dimensions, fewer correlated samples.
 - Implemented in **Stan**, **PyMC**, etc.

Obtaining brute force posterior distribution

- Many different types of Markov Chain Monte Carlo (MCMC):
 - **No-U-Turn Sampler (NUTS):**
 - An extension of HMC that automatically tunes parameters.
 - Avoids manual tuning of step size and trajectory length.
 - Used in **Stan** and **PyMC**
- Also non-MCMC:
 - **Variational Inference (VI):**
 - Optimizes a simpler distribution to approximate the posterior.
 - Much faster than MCMC, but gives an **approximation**, not exact samples.
 - Useful in large-scale models (e.g., deep learning).
 - **Laplace Approximation (LA):**
 - More on that later...

Sampling posterior

- Example basic linear multi-predictor regression:

$$y_i \sim \text{Normal}(\mu_i, \sigma) \quad i = 1, \dots, N$$

$$\mu_i = \alpha + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$$

$$\alpha = 0$$

$$\beta_1 = 0.2$$

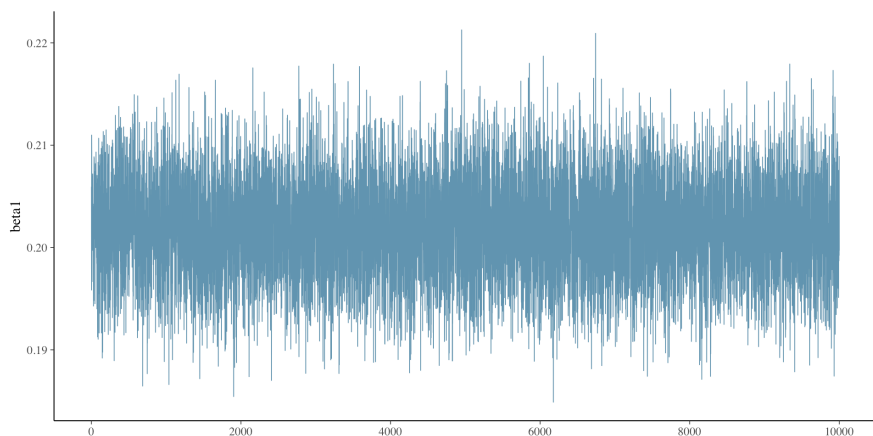
$$\beta_2 = 0.5$$

$$\beta_3 = 0.3$$

$$\sigma = 0.5$$

38

Idealized ‘hairy caterpillar’



39

Informative or non-informative priors?

- A source of intrigue from pre-Bayesians is how priors are chosen.
- Priors can be informed by existing knowledge.
- But what if we don't know anything really prior to inference?
- Most of the time we will not have analytical solutions.

How to estimate non-conjugate posterior?

- Some kind of software to help us implement models for inference.
- Using [R](#).
- Packages ([BUGS](#), [NIMBLE](#), [STAN](#), [R-INLA](#) etc.).
- We will be focusing on using [NIMBLE](#) and [R-INLA](#) throughout the three days.

Why we're using [NIMBLE](#) for the start of the workshop

- Interpretability.
- Flexibility.

Using [NIMBLE](#) for Bayesian inference

Illustrative example of basic regression in [NIMBLE](#)

- Let's go through an example from the upcoming lab.

Linear regression

- Example basic linear multi-predictor regression:

$$\begin{aligned}
 y_i &\sim \text{Normal}(\mu_i, \sigma) \quad i = 1, \dots, N \\
 \mu_i &= \alpha + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 \\
 \alpha &= 0 \\
 \beta_1 &= 0.2 \\
 \beta_2 &= 0.5 \\
 \beta_3 &= 0.3 \\
 \sigma &= 0.5
 \end{aligned}$$

Linear regression

- What does the dataset look like?

y	x1	x2	x3
-0.15	-0.62	-0.80	0.23
0.02	0.19	-1.06	0.23
-1.09	-0.83	-1.04	-0.65
-1.00	1.61	-1.19	-1.94
-0.14	0.34	-0.50	1.03
0.34	-0.81	-0.52	-0.29

Linear regression

- First create some example data for our model:

```

1 set.seed(1)
2 p <- 3 # number of explanatory variables
3 n <- 10000 # number of observations
4 X <- matrix(round(rnorm(p * n), 2), nrow = n, ncol = p) # explanatory variables
5 true_betas <- c(0.2, 0.5, 0.3) # coefficients for beta1, beta2, beta3
6 sigma <- 0.5 # standard deviation of variability around perfect agreement
7 y <- rnorm(n, X %*% true_betas, sigma)
8
9 # centre the explanatory variables (shifts the intercept, leaves slopes alone)
10 x1 <- X[, 1] - mean(X[, 1])
11 x2 <- X[, 2] - mean(X[, 2])
12 x3 <- X[, 3] - mean(X[, 3])

```

Linear regression

- What does equivalent frequentist model output look like for reference?

```

1 model_freq <- lm(
2   y ~ x1 + x2 + x3,
3   data = df
4 )
5 central_est <- t(t(model_freq$coefficients))
6 conf_int <- confint(model_freq)
7 cbind(central_est, conf_int)

```

Linear regression

- What does equivalent frequentist model output look like for reference?

	Estimate	2.5%	97.5%
(Intercept)	-0.004	-0.014	0.006
x1	0.202	0.192	0.212
x2	0.493	0.483	0.503
x3	0.292	0.282	0.301

Linear regression

- Setting up data (will explain more in lab)

```

1 constants <- list(n = n)
2 data <- list(y = y, x1 = x1, x2 = x2, x3 = x3)
3
4 # one set of starting values per chain, deliberately spread out
5 inits <- list(
6   list(alpha = -1, beta1 = -1, beta2 = -1, beta3 = -1, sigma = 0.5),
7   list(alpha = 0, beta1 = 0, beta2 = 0, beta3 = 0, sigma = 1),
8   list(alpha = 1, beta1 = 1, beta2 = 1, beta3 = 1, sigma = 2),
9   list(alpha = 2, beta1 = -2, beta2 = 2, beta3 = -2, sigma = 3)
10 )

```

Linear regression

- NIMBLE** adopts and extends **BUGS** as a modeling language and lets you program with the models you create.
- Looks like below (as seen earlier in lecture and later in lab).

```

1 library(nimble)
2 code <- nimbleCode({
3   # priors for parameters
4   alpha ~ dnorm(0, sd = 100) # prior for alpha
5   beta1 ~ dnorm(0, sd = 100) # prior for beta1
6   beta2 ~ dnorm(0, sd = 100) # prior for beta2
7   beta3 ~ dnorm(0, sd = 100) # prior for beta3
8   sigma ~ dunif(0, 100) # prior for variance components
9
10  # regression formula
11  for (i in 1:n) { # n is the number of observations we have in the data
12    mu[i] <- alpha + beta1 * x1[i] + beta2 * x2[i] + beta3 * x3[i] # manual entry of linear predictors
13    y[i] ~ dnorm(mu[i], sd = sigma)
14  }
15 })

```

Linear regression

- Run the MCMC simulations. There are lots of arguments in the main function, `nimbleMCMC()`.

```

1 tic <- Sys.time()
2 nimbleMCMC_samples_initial <- nimbleMCMC(
3   code = code,
4   data = data,
5   constants = constants,
6   inits = inits,
7   niter = 2000, # run 2000 samples per chain
8   nburnin = 1000, # discard the first 1000 as warm-up
9   nchains = 4, # four chains, each from different starting values
10  setSeed = 1,
11  samplesAsCodaMCMC = TRUE,
12  progressBar = FALSE
13 )
14
15 toc <- Sys.time()
16 toc - tic

```

Linear regression

- Run the MCMC simulations. There are lots of arguments in the main function, nimbleMCMC().

Runtime

131.0 seconds

Linear regression

- The following operations will help us to answer: What is the summary of each estimated parameter from the samples?

variable	mean	median	sd	mad	q5	q95
alpha	-0.004	-0.004	0.005	0.005	-0.012	0.004
beta1	0.202	0.202	0.005	0.005	0.194	0.210
beta2	0.493	0.493	0.005	0.005	0.484	0.501
beta3	0.292	0.291	0.005	0.005	0.283	0.300
sigma	0.499	0.499	0.004	0.003	0.494	0.505

Linear regression

- The following operations will help us to answer: What is the summary of each estimated parameter from the samples?

```
1 library(bayesplot)
2 library(posterior)
3 summarise_draws(nimbleMCMC_samples_initial, default_summary_measures())
```

Linear regression

- Then examine how well the model has converged, which typically is identified by how close each rhat value is to 1.00.
- rhat compares variation within each chain to variation between the four chains.

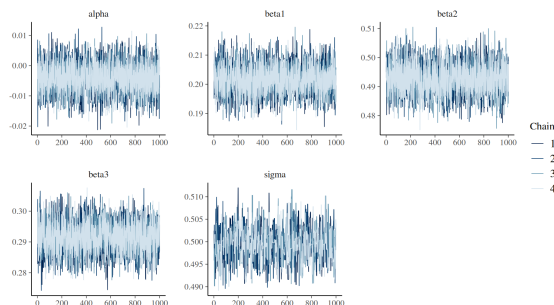
```
1 summarise_draws(nimbleMCMC_samples_initial, default_convergence_measures())
```

Linear regression

variable	rhat	ess_bulk	ess_tail
alpha	1.001	4090.674	4088.974
beta1	0.999	3663.781	3990.647
beta2	1.000	4125.737	3686.433
beta3	1.000	4104.405	3979.314
sigma	1.005	878.279	963.391

Linear regression

- What do the samples of the target parameters actually look like?
- One colour per chain: we want them overlapping, not drifting apart.



Linear regression

- What do the samples of the target parameters actually look like?
- One colour per chain: we want them overlapping, not drifting apart.

```
1 mcmc_trace(nimbleMCMC_samples_initial)
```

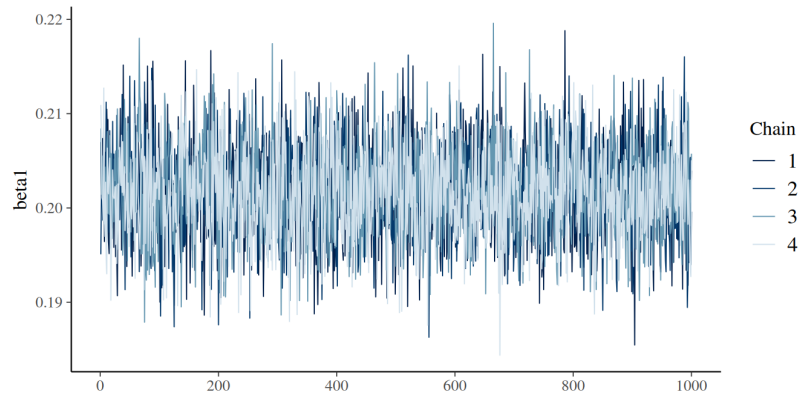
Linear regression

Let's now focus on **beta1** (which we know is 0.2 from setting up initially).

```
1 mcmc_trace(nimbleMCMC_samples_initial, pars = c("beta1"))
```

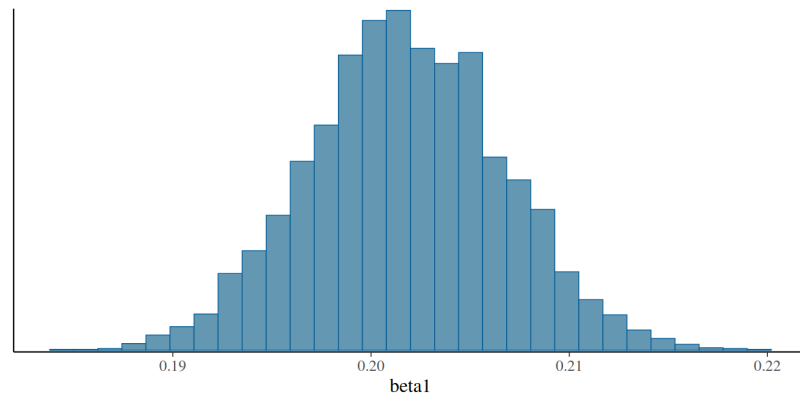

Linear regression

Let's now focus on `beta1` (which we know is 0.2 from setting up initially).



Linear regression

Plotting `beta1` samples (posterior) as a histogram, pooled across all four chains...



Linear regression

Plotting `beta1` samples (posterior) as a histogram, pooled across all four chains...

```
1 mcmc_hist(nimbleMCMC_samples_initial, pars = c("beta1"))
```

Getting ready for the lab

The lab for this session

- This lab will involve taking some models and concepts from the Introduction to Bayesian Methods lecture and introduce you to the way **NIMBLE** works.
- During this lab session, we will:
 1. Explore how **NIMBLE** is written and works;
 2. Write some common regression models (Normal, logistic regression, Poisson);
 3. Understand how basic model assessment is made; and
 4. Test out how adjustments of models are made via different priors and more samples.

Questions?